Day 4

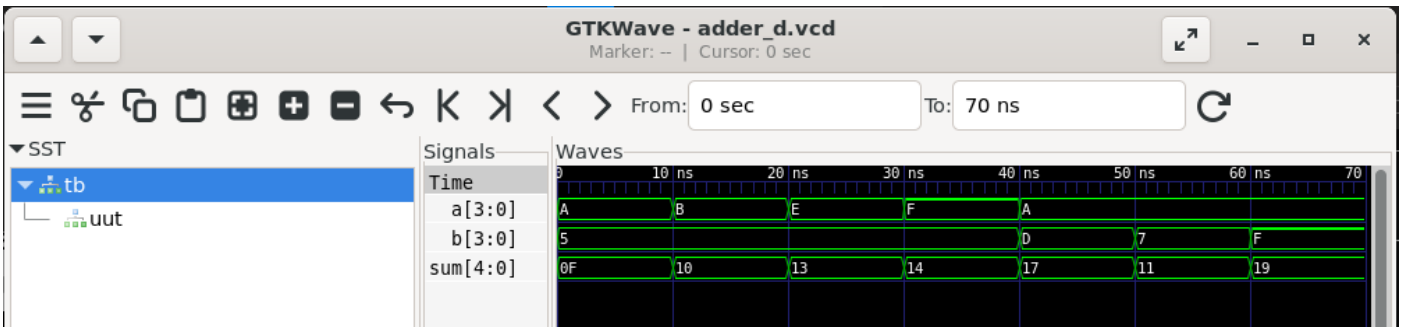
4 bit adder

Verilog Code:

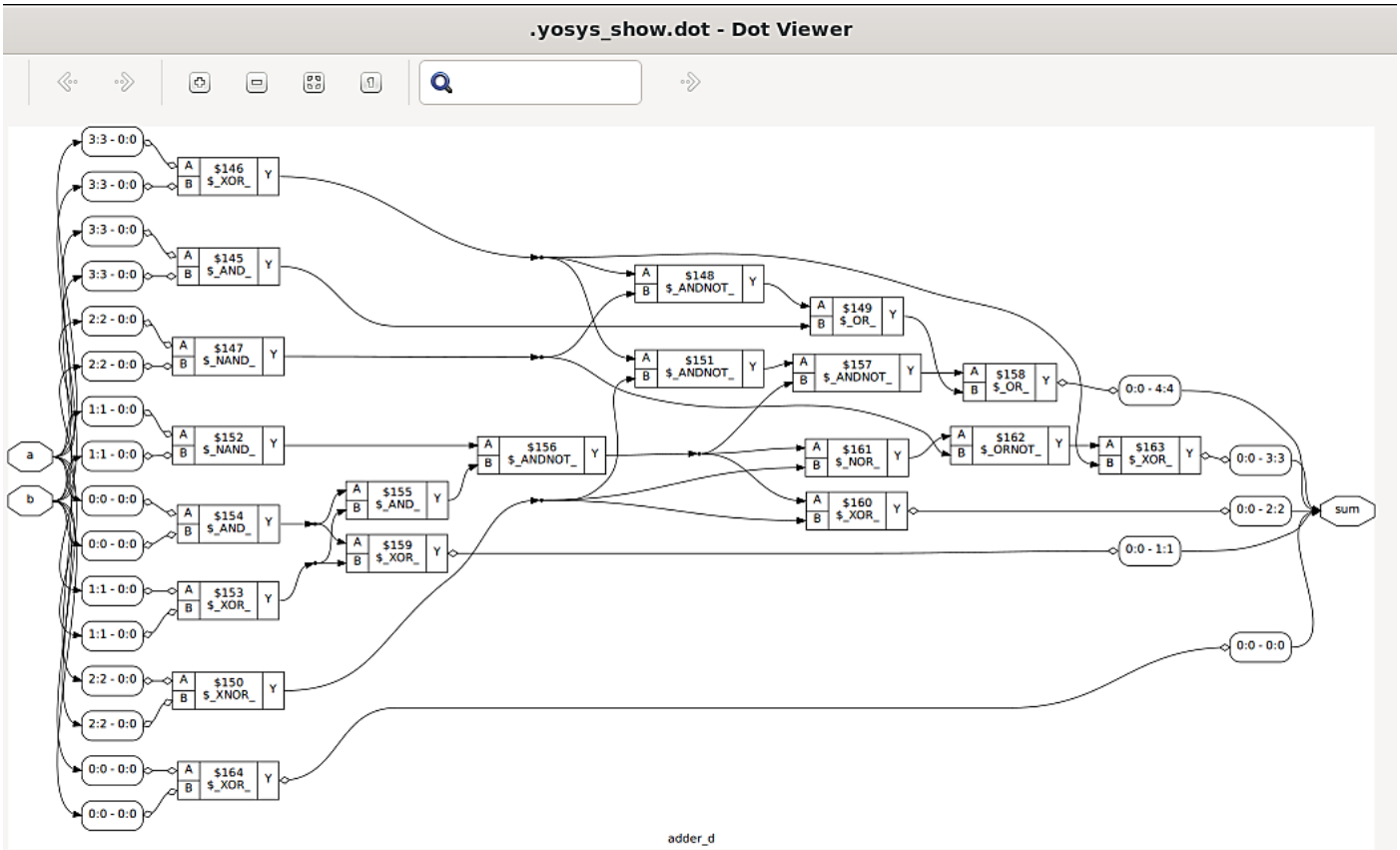
```
module adder_d #(parameter N = 8)(
 input [N-1:0] a, b,
 output [N:0] sum);

 assign sum = a + b;

endmodule
```

Testbench:

```
`timescale 1ns/1ps

module tb;
reg [3:0] a, b;
wire [4:0] sum;

adder_d uut(.a(a), .b(b), .sum(sum));
//initial begin
//clk = 0;
//forever #5 clk = ~clk ;
//end
initial begin
$dumpfile("adder_d.vcd");
$dumpvars(0, tb);
end

initial begin
 a = 4'b0010; b = 4'b0101; #10;
 a = 4'b0011; b = 4'b0101; #10;
 a = 4'b0110; b = 4'b0101; #10;
 a = 4'b0111; b = 4'b0101; #10;
 a = 4'b1010; b = 4'b1101; #10;
 a = 4'b1010; b = 4'b0111; #10;
 a = 4'b1010; b = 4'b1111; #10;
 a = 4'b1100; b = 4'b0011; #10;
 $finish;
 end
 endmodule
```



Simulation and Synthesis output:



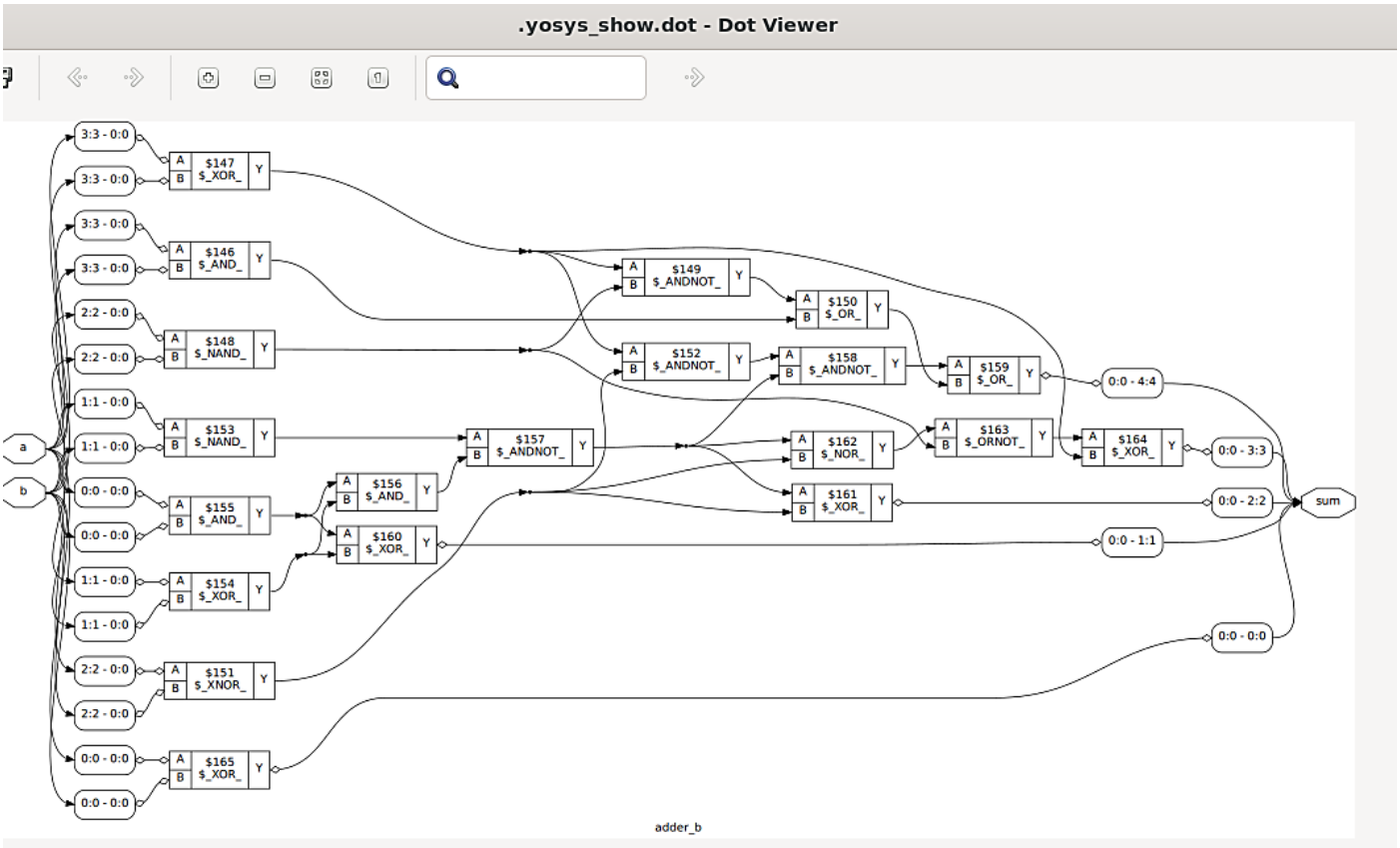
2.26. Printing statistics.

=== adder_d ===

```
   Number of wires:                 18
   Number of wire bits:             28
   Number of public wires:           3
   Number of public wire bits:      13
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                 20
     $_ANDNOT_                       4
     $_AND_                          3
     $_NAND_                         2
     $_NOR_                          1
     $_ORNOT_                        1
     $_OR_                           2
     $_XNOR_                         1
     $_XOR_                          6
```

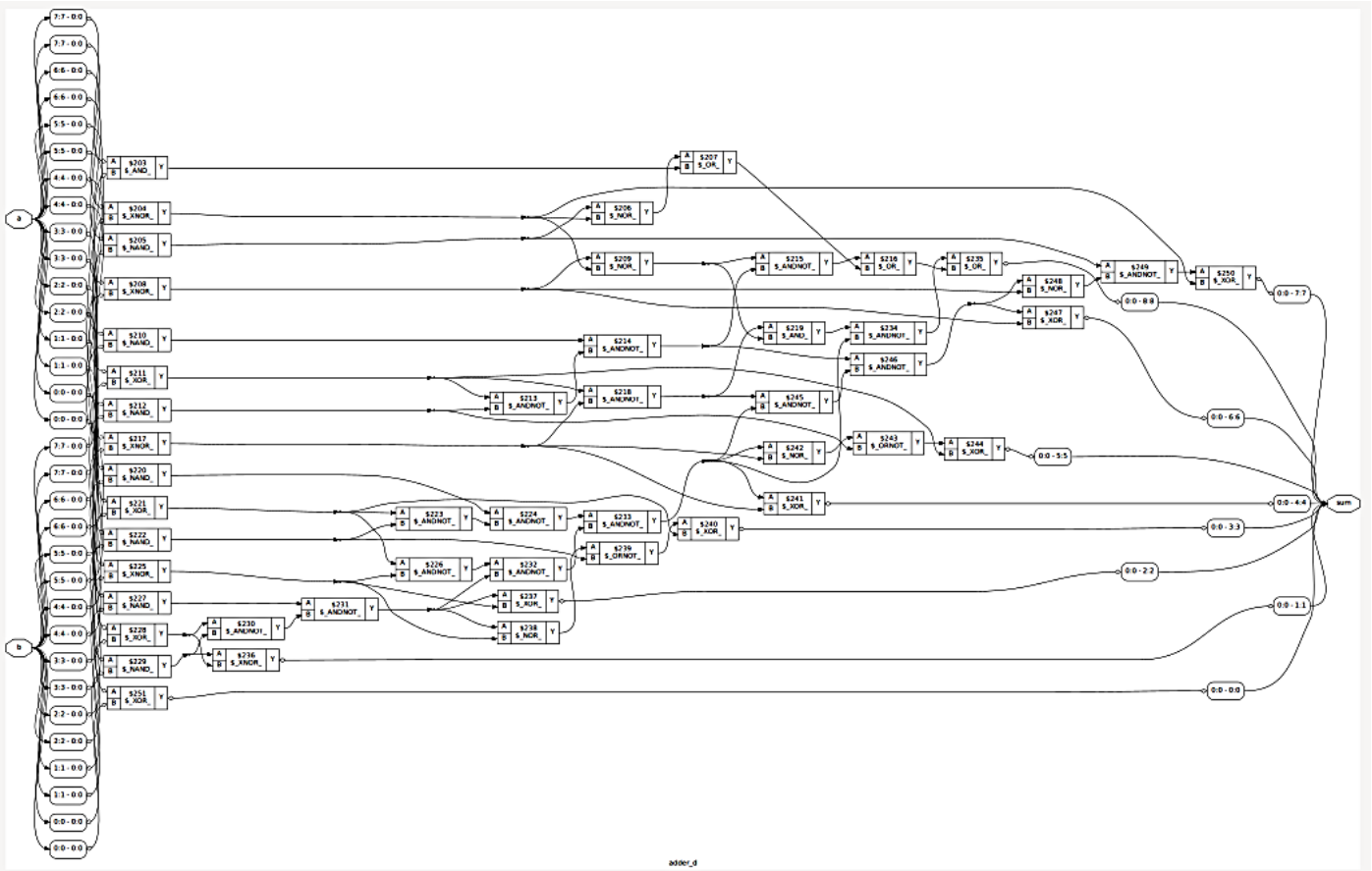
4-bit adder using behavioural modelling

```
module adder_b #(parameter N = 4)(
 input [N-1:0] a, b,
 output reg [N:0] sum);

always @(*) begin
 sum <= a + b;
 end
endmodule
```



Note: We can see that for both dataflow and behavioural modelling of the 4-bit adder, the synthesised hardware is same, with equal number of cell counts

8-bit adder



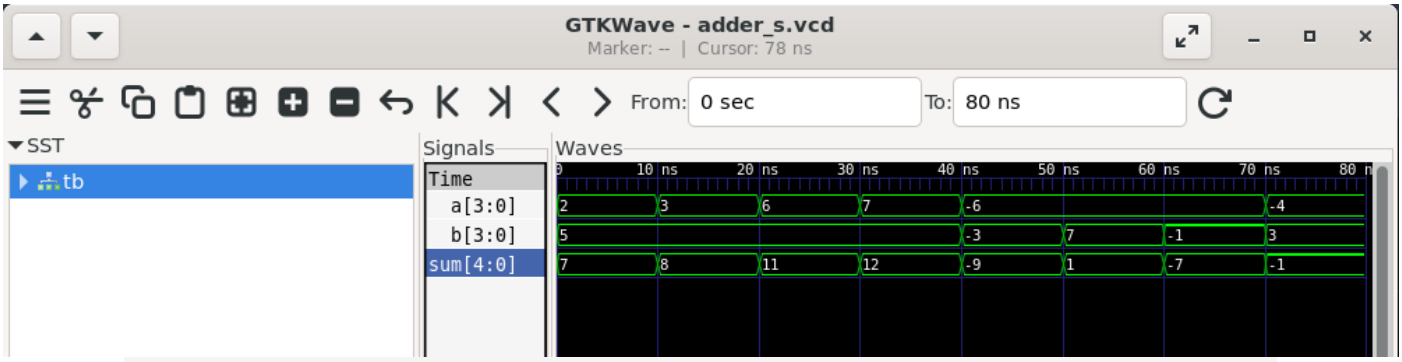
2.26. Printing statistics.

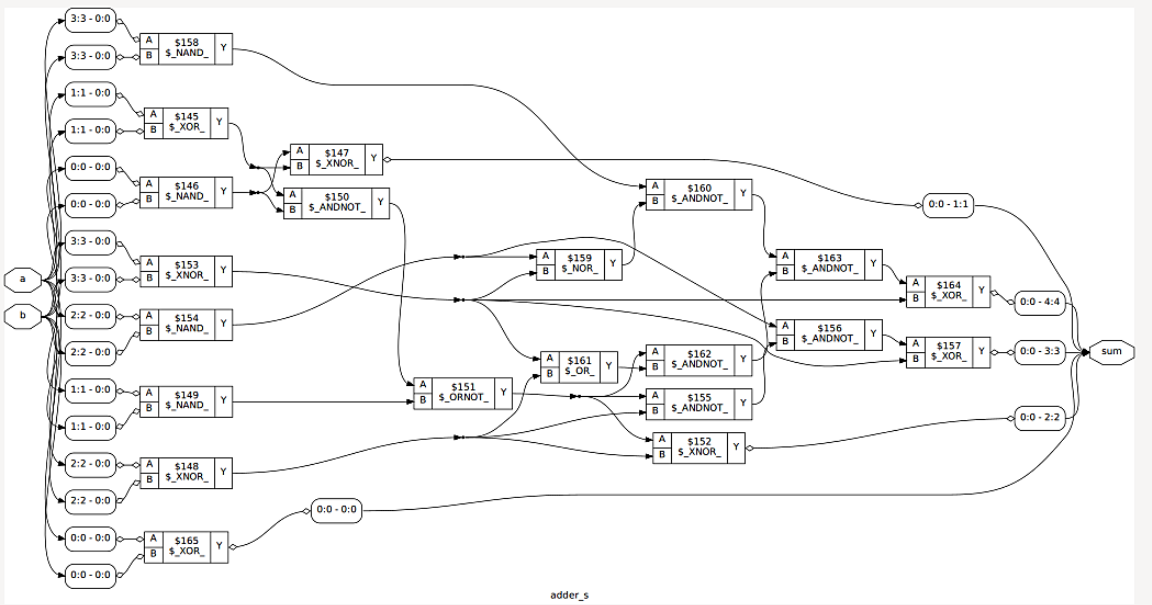
=== adder_d ===

```
   Number of wires:                 43
   Number of wire bits:             65
   Number of public wires:           3
   Number of public wire bits:      25
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                 49
     $_ANDNOT_                      15
     $_AND_                          2
     $_NAND_                         7
     $_NOR_                          5
     $_ORNOT_                        2
     $_OR_                           3
     $_XNOR_                         5
     $_XOR_                         10
```

4-bit signed addition:

```
module adder_s(
 input signed [3:0] a, b,
 output signed [4:0] sum);

assign sum = a + b;
endmodule
```





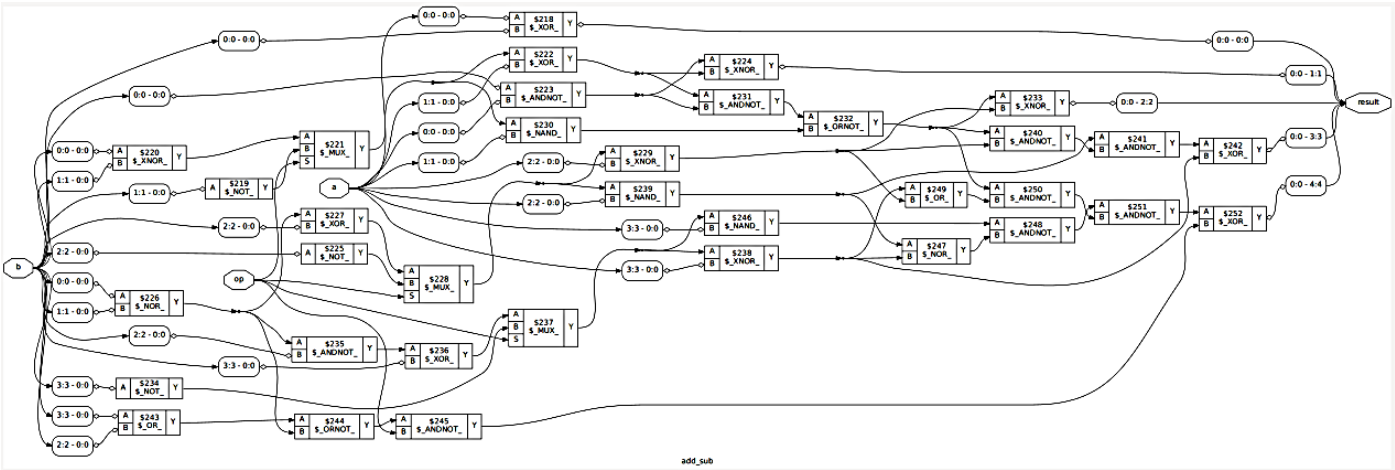
2.26. Printing statistics.

=== adder_s ===

```
   Number of wires:                 19
   Number of wire bits:             29
   Number of public wires:           3
   Number of public wire bits:      13
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                 21
     $_ANDNOT_                       6
     $_NAND_                         4
     $_NOR_                          1
     $_ORNOT_                        1
     $_OR_                           1
     $_XNOR_                         4
     $_XOR_                          4
```

4-bit Add-sub module

```
module add_sub(
   input      [3:0] a, b,
   input          op,   // 0=add, 1=sub
   output reg [4:0] result
);
   always @(*) begin
      if(op) result = a - b;
      else   result = a + b;
   end
endmodule
```



2.26. Printing statistics.

=== add_sub ===

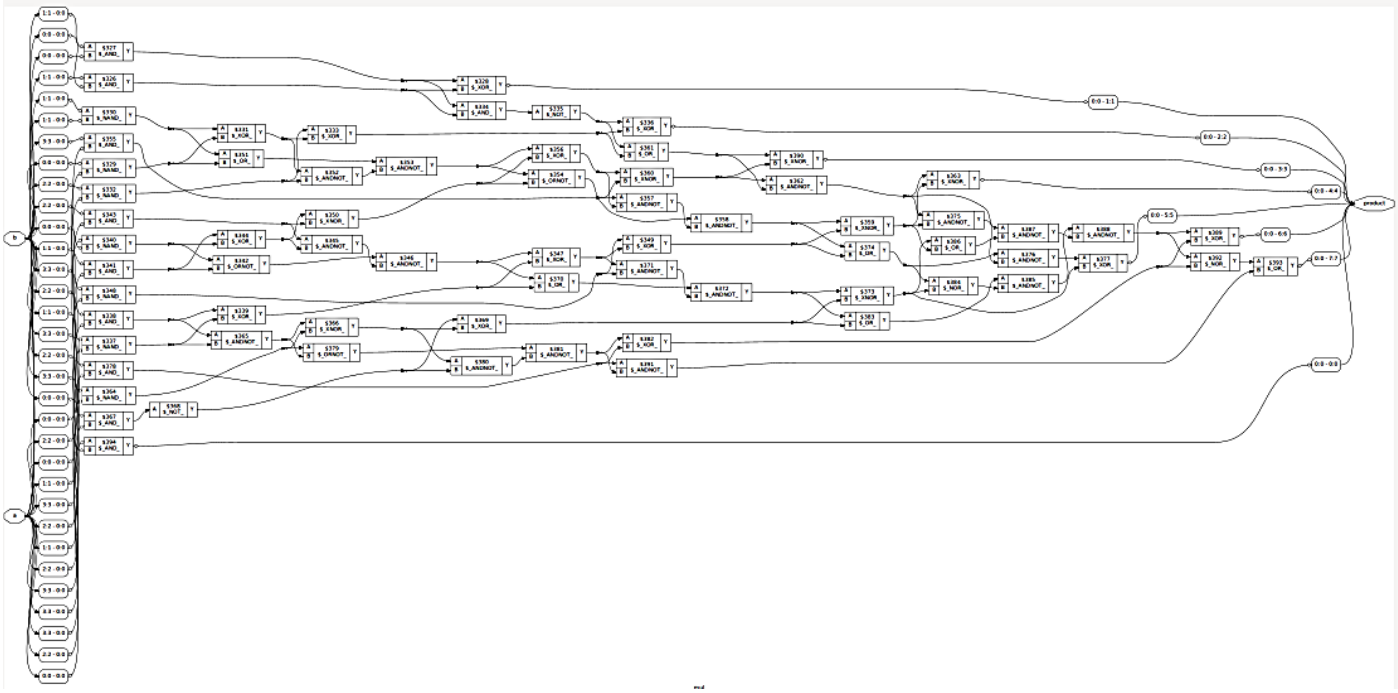
```
   Number of wires:                 34
   Number of wire bits:             44
   Number of public wires:           4
   Number of public wire bits:      14
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
```

Number of cells: 35
$_ANDNOT_ 9
$_MUX_ 3
$_NAND_ 3
$_NOR_ 2
$_NOT_ 3
$_ORNOT_ 2
$_OR_ 2
$_XNOR_ 5
$_XOR_ 6

Note: We can see that there's no separate 2 adder blocks here, the add and sub operations are decided by the mux, and the subtraction operation isn't actually A-B, it's A+B(bar)+1.

Multipliers:

```
module mul(
   input  [3:0] a, b,
   output [7:0] product
);
   assign product = a * b;
endmodule
```

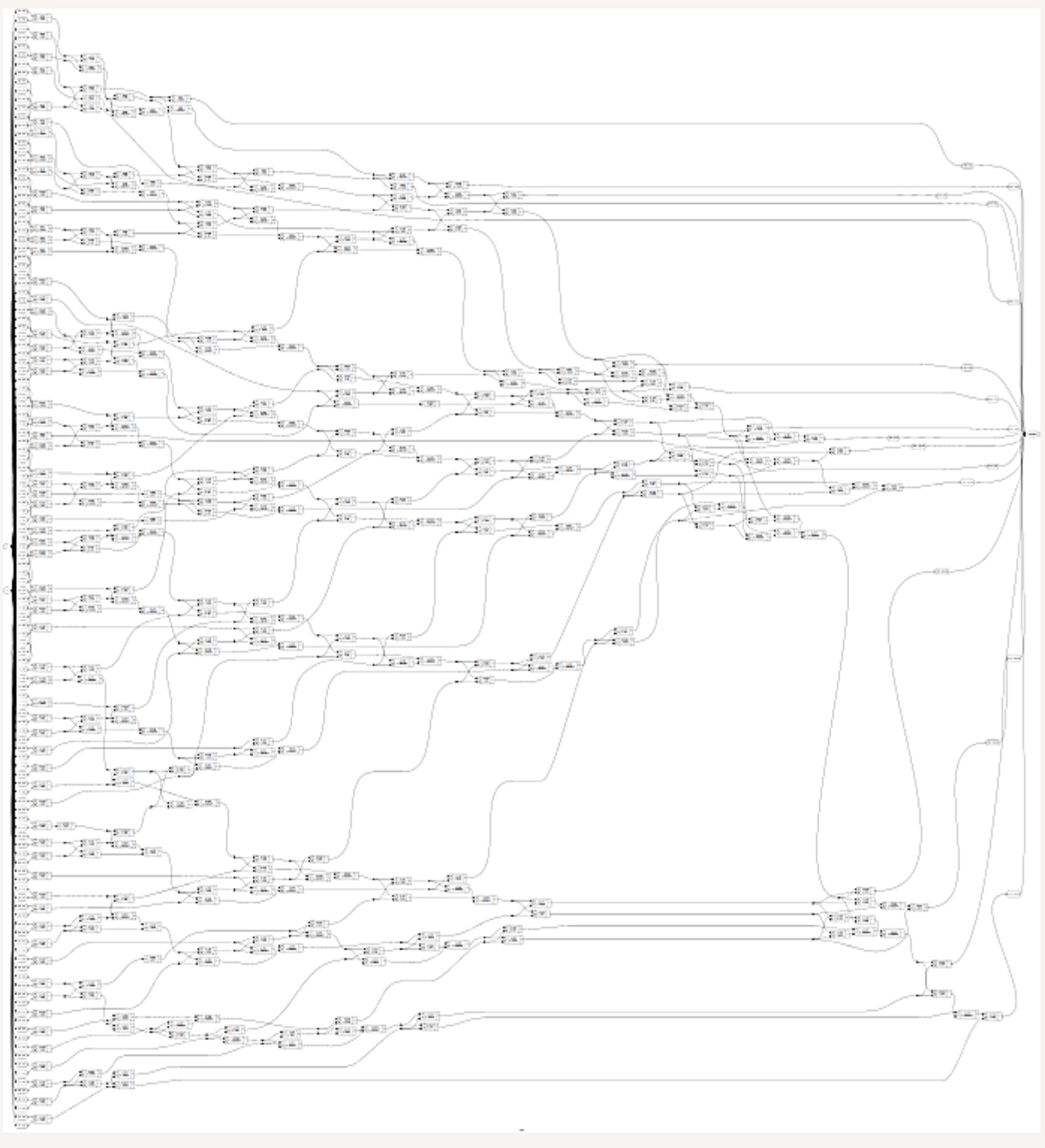


2.26. Printing statistics.

=== mul ===

Number of wires: 64
Number of wire bits: 77
Number of public wires: 3
Number of public wire bits: 16
Number of memories: 0
Number of memory bits: 0
Number of processes: 0
Number of cells: 69
$_ANDNOT_ 18
$_AND_ 10
$_NAND_ 7
$_NOR_ 2

```
    $_NOT_                    2
    $_ORNOT_                  3
    $_OR_                     7
    $_XNOR_                   7
    $_XOR_                   13

module mul(
   input  [7:0] a, b,
   output [15:0] product
);
   assign product = a * b;
endmodule
```



2.26. Printing statistics.

```
=== mul ===

   Number of wires:                 341
   Number of wire bits:             370
   Number of public wires:            3
   Number of public wire bits:       32
   Number of memories:                0
   Number of memory bits:             0
   Number of processes:               0
   Number of cells:                 354
     $_ANDNOT_                       97
     $_AND_                          55
     $_NAND_                         25
     $_NOR_                          12
     $_NOT_                           7
     $_ORNOT_                        10
     $_OR_                           37
     $_XNOR_                         25
     $_XOR_                          86
```

Note: As the number of the bits to be multiplied is increasing, the hardware's size is increasing too, for this reason current modern processors use a special DSP (Digital Signal Processing) block to perform these complex operations.